

TTFL: Dual-Timescale Trust Flow for TEE-Anchored Federated Learning under Non-IID Data

Jiakai Zhou, Chao Bu

*School of Computer Science and Engineering
Tianjin University of Technology, Tianjin, China
Email: wllsp0@stud.tjut.edu.cn, bc_0722@163.com*

Abstract—Federated learning (FL) enables collaborative edge intelligence without moving raw data, but open participation creates a trust-continuity problem: a client may execute an admissible training process while producing a harmful update, and a benign long-tail client may look anomalous under Non-IID data. This paper presents TTFL as a trust-flow framework that assigns different evidence to different decisions. Execution evidence establishes an admission boundary, privacy-aware data and runtime checks provide limited statistical evidence, and the cloud server audits update content and cross-round behavior before assigning layer privileges. The central design is a temporal separation between long-term utility and short-term risk: a persistent contribution signal supports retention, whereas a current risk signal can restrict an update without immediately erasing a client’s history. A conservative cold-start policy, explicit state transitions, and risk-gated data-size-aware re-normalization complete the decision loop. The evaluation uses an Intel TDX-backed client execution path with a paired host configuration, Flower/PyTorch and CIFAR-10, five seeds per configuration, 20 admitted clients, six attack behaviors, and a 30% malicious setting. Targeted ASR is macro-averaged over the three targeted behaviors, while untargeted attacks are evaluated with clean accuracy, loss, and state-transition metrics. TTFL reaches 92.31% clean accuracy, 10.21% targeted ASR under the stated protocol, and no permanent false ban in the tested runs.

Index Terms—Federated learning, edge computing, trusted execution environment, risk modeling, layered aggregation, backdoor defense

1. Introduction

FL exchanges model parameters so that edge devices can contribute without moving their raw samples [1], [2]. An open edge deployment must answer three separate questions. The first concerns execution: did the client follow an admissible procedure? The second concerns the submitted update: does it pass the data and model checks that are available? The third concerns time: has the observed behavior persisted long enough to affect later participation? Non-IID data couples these questions in an inconvenient way. A long-tail client can repeatedly point away from a reference direction,

while an attacker can behave normally for several rounds and then inject a damaging update. Collapsing all three observations into one score either removes benign clients too quickly or leaves a current attack unchecked.

Existing defenses use geometric, statistical, reference-based, or temporal evidence to flag unusual updates [3], [4], [5], [6], [7]. Backdoor-oriented methods additionally examine model behavior, learned representations, clusters, and collusion [8], [9], [10], [11]. Such tests say little about how local training was executed, and the same tests can interpret Non-IID drift as an attack. TEE-assisted FL contributes a hardware-rooted execution boundary [12], [13], [14], [15]; admission based on that boundary still leaves the semantic safety of an admitted update unresolved.

The proposed *Trust Flow* policy keeps two time scales visible during participation decisions. A client with weak historical utility but little current risk remains eligible to recover; a client with strong history but a current high-risk update is restricted despite its earlier record. The policy is therefore a decision contract. Execution evidence determines admission, current evidence gates the submitted update, and persistence rules control later privileges. Intel TDX supplies the execution boundary in which this contract is evaluated. The method is characterized by the assignment of heterogeneous evidence to distinct state transitions and aggregation layers, rather than by a single new probe or a stand-alone robust aggregator. The contributions are:

- 1) A dual-stream state representation gives historical contribution and short-term risk separate roles, preserving benign-client retention while allowing a rapid response to attacks.
- 2) A state-to-aggregation policy links recoverable suspicion, persistent-risk escalation, and layer-specific weighting. Content quality can lower a contribution without, by itself, forcing irreversible exclusion.
- 3) An empirical analysis relates the single-stream and HistPerf-only ablations to the retention–detection tradeoff, and separates the effects of risk probes from those of weight re-normalization.

2. Background and Related Work

Federated learning in edge computing. Edge deployment leaves a continuity gap between a valid execution and

a safe model update. A client may satisfy the first condition yet submit an unsafe update, while a benign long-tail client may look unusual. Execution evidence and update evidence must remain distinguishable before aggregation.

Poisoning and backdoor attacks. Sign flipping and scaling damage optimization directly. Trigger, clean-label, and semantic backdoors instead aim to implant targeted errors while keeping ordinary predictions plausible. Their updates can remain close to an expected direction [16], [17]; intermittent attackers may also accumulate a favorable history before acting.

Active defenses and their limits. Krum and Trimmed Mean remove geometric outliers [3], [4], FLTrust weights updates against a clean direction [5], and FoolsGold and FLDetector use cross-client or cross-round evidence [6], [7]. DeepSight, FLAME, CrowdGuard, RoseAgg, FLPurifier, ShieldFL, and RaSA inspect representations, backdoor behavior, collusion, or privacy-preserving updates [8], [9], [10], [11], [18], [19], [20]. TMT-FL and related systems connect execution integrity to model training [15]. These approaches observe different evidence spaces. Trust Flow places an interface between them: attestation determines execution eligibility, privacy-aware checks expose bounded statistics, content probes inspect the current update, and the two temporal streams decide future privileges. One observation can therefore alter a layer weight, a recoverable state, or a persistent state only when the corresponding rule is satisfied. The experiments use classical methods as matched aggregation baselines and stateful variants to isolate this temporal interface.

TEE-assisted trusted training. TEEs protect execution and key material and have been used to support FL integrity and attack mitigation [12], [13], [14], [15], [21]. A quote authenticates the measured path, freshness, and update provenance. TTFL combines that hardware evidence with privacy-aware verification and cloud-side semantic auditing before assigning aggregation privileges.

3. System Model and Threat Assumptions

The system contains a central server $\mathcal{S}$ and K edge clients $\mathcal{C} = \{c_1, \dots, c_K\}$. In round t , admitted clients receive $W^{(t-1)}$, train locally, upload $\Delta W_k^{(t)}$, and the server updates $W^{(t)} = W^{(t-1)} + \text{Agg}(\{\Delta W_k^{(t)}\})$. The global objective is

$$\min_W \mathcal{L}(W) = \sum_{k=1}^K p_k \mathcal{L}_k(W), \quad p_k = |\mathcal{D}_k| / \sum_j |\mathcal{D}_j|. \quad (1)$$

The edge workflow is divided between client, edge, and cloud roles. Local training produces an attestation-bound report. A privacy-aware verification layer reveals only the declared data and runtime statistics required for admission and audit; raw samples stay on the client. The server examines the update and the derived evidence, advances the temporal states, and carries out trust-weighted aggregation. This arrangement differs from cryptographic secure aggregation: per-client evidence must remain available to the server

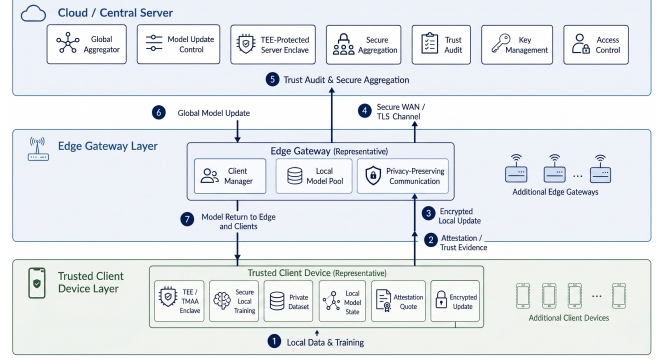


Figure 1. TEE-anchored client-edge-cloud architecture.

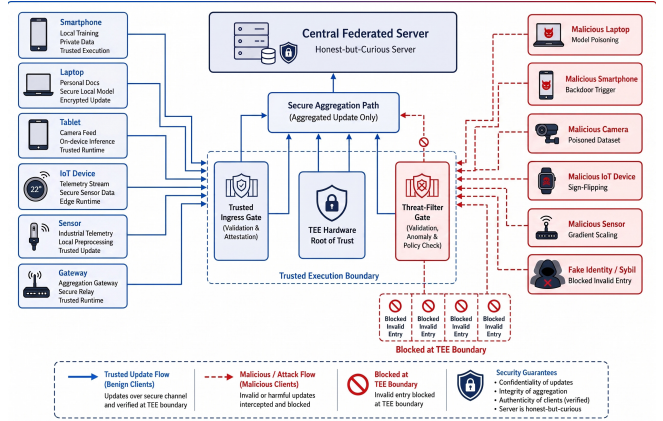


Figure 2. Composite poisoning and penetration threat model.

for the trust-flow decision, while the verification path limits what is disclosed by the client.

Threat model. The server is honest-but-curious and follows the aggregation protocol. It can see individual updates and audit statistics, but it does not change the stated state-transition rules. An admitted client may alter labels, samples, or optimization objectives and may conduct label flipping, trigger backdoors, clean-label poisoning, semantic backdoors, sign flipping, or gradient scaling. In the deployed Intel TDX path, the client runs inside a measured confidential virtual machine. Its TDX evidence binds a verifier nonce, round identifier, and update digest to the measured workload. The boundary protects the measured path, device keys, report integrity and freshness, and the update binding against a compromised host. Privacy-aware verification and cloud-side content and temporal audits complement this hardware evidence by checking selected declared statistics and model-level behavior without exposing raw samples. Data- or objective-level poisoning through a valid measured process remains in scope; host tampering after attestation does not. A failed report or update-binding check is rejected during admission, after which valid evidence enters the content and temporal stages. Physical probing and advanced microarchitectural side channels are excluded. The analytical assumption is fewer than 50% malicious admitted clients;

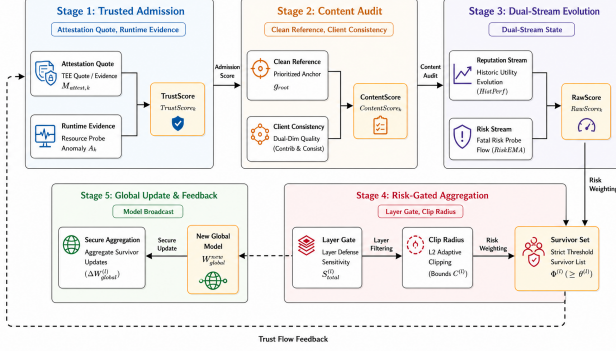


Figure 3. Overview of the four-stage trust-flow defense loop.

the 50% experiment is an out-of-assumption stress test, not a Byzantine threshold or a theoretical guarantee. The server-held clean proxy initializes content auditing, while the trusted-peer branch provides the corresponding operation when no proxy is available.

Design goals. Trust Flow is designed to maintain convergence under the stated attacks, avoid permanent false bans of long-tail clients, and keep the assurance boundary explicit. Execution, current content, historical utility, and recent risk retain separate meanings until layer-wise aggregation. The reported TrustReport size is about 15 KB, and server aggregation plus audit increases from 2.10 s to 4.85 s. The auxiliary security path measures approximately 11 ms per admitted-client round, while device-side TEE Quote generation, signing, and lightweight monitoring measure approximately 12–15 ms per admitted-client round. These component measurements are reported separately from server processing and end-to-end communication.

4. Trust-Flow Framework

Each FL round passes through four linked stages: admission and runtime assessment, reference-based content auditing, dual-stream state evolution, and risk-gated layered aggregation. Figure 3 presents the loop.

The stages produce *TrustScore*, *ContentScore*, the slow *HistPerf* and fast *RiskEMA* states, and finally *RawScore*. Invalid proofs are rejected first. Privacy-aware statistics and the current update are then audited, temporal state is advanced, and layer privileges are assigned. Cold start follows the same order: a new client receives a provisional first-round privilege, current evidence may reduce it, and irreversible transitions require accumulated evidence. A single low content score can therefore be recovered by a Non-IID client, whereas persistent or immediately strong risk can restrict its update.

4.1. Admission and Content Auditing

Admission begins with a TEE check. Let $M_{\text{attest},k} \in \{0,1\}$ indicate whether the report, freshness nonce, and binding to the round and update digest are valid. During

initialization, the client invokes a device-unique key and creates a remote-attestation quote containing the vendor measurement digest, code identity, runtime configuration, round nonce, update digest, and monotonic freshness counter. Communication privileges are granted only after this check. During local training, the trusted management agent records the standardized sequence $\mathcal{X}_k = \{\Delta\text{grad}, \Delta\text{loss}, \text{instr_dist}\}$. The normalized entropy $h_k^{(t)} = H(\mathcal{X}_k^{(t)})/H_{\text{max}} \in [0,1]$ is computed after concatenating the three standardized feature histograms; H_{max} denotes the entropy of their uniform concatenation. The implementation uses

$$\begin{aligned} R_k^{(t)} &= \exp(\lambda h_k^{(t)}), \\ Z_k^{(t)} &= \text{clip}(w_a M_{\text{attest},k} \\ &\quad + (1 - w_a)(1 - h_k^{(t)}), 0, 1), \end{aligned} \quad (2)$$

where $w_a = 0.7$ and $\lambda = 1.0$. High entropy is therefore treated as uncertain evidence, not as a direct maliciousness label.

The trust state follows a scalar Kalman filtering view. It is updated as

$$T_k^{-(t)} = T_k^{(t-1)}, \quad P_k^{-(t)} = P_k^{(t-1)} + Q, \quad (3)$$

$$\begin{aligned} K_k^{(t)} &= \frac{P_k^{-(t)}}{P_k^{-(t)} + R_k^{(t)}}, \\ T_k^{(t)} &= T_k^{-(t)} + K_k^{(t)}(Z_k^{(t)} - T_k^{-(t)}), \\ P_k^{(t)} &= (1 - K_k^{(t)})P_k^{-(t)}. \end{aligned} \quad (4)$$

The observation $Z_k^{(t)} \in [0,1]$ is defined above. Here $Q = 0.01$ is the fixed process-noise variance and $R_k^{(t)}$ is the entropy-derived observation variance. A fresh admitted client starts with $T_k^{(0)} = 1$, $P_k^{(0)} = 0.25$, $RiskEMA_k^{(0)} = 0$, and $M_{\text{attest},k} = 1$. The resulting privilege factor is

$$TrustScore_k^{(t)} = M_{\text{attest},k} \text{clip}\left(\frac{T_k^{(t)}}{1 + P_k^{(t)}}, 0, 1\right). \quad (5)$$

High entropy increases $R_k^{(t)}$ and lowers the Kalman gain. An uncertain observation thus cannot rewrite cross-round trust in one step.

The server next constructs a clean guiding direction. When a small clean validation gradient exists, $g_{\text{root}} = g_{\text{root_clean}}$. Otherwise, admitted clients with high trust and low previous risk are used with reference weights $\omega_k^{\text{ref}} = TrustScore_k^{(t)}(1 - RiskEMA_k^{(t-1)})^2$. In the reported experiments, the server holds a balanced 500-sample proxy set, with 50 images per CIFAR-10 class, and this set is excluded from client training. The fixed proxy partition is part of the common experiment protocol. Methods with a reference channel use it for their defined reference operation, whereas methods without that channel retain their own aggregation rule under the same data, initialization, and communication budget. The proxy gives content auditing a stable starting direction. Peer consistency and temporal states continue to add evidence after initialization. If the proxy is unavailable,

the trusted-peer branch builds the corresponding reference from admitted updates. For each update, content quality combines reference consistency and peer consistency:

$$S_{contrib,k} = \max(0, \alpha_1 + \alpha_2 \cos(\Delta W_k, g_{root})), \quad (6)$$

$$S_{consist,k} = \frac{1}{\sum_{j \neq k} TrustScore_j + \varepsilon} \times \sum_{j \neq k} [TrustScore_j \times \max(0, \alpha_1 + \alpha_2 \cos(\Delta W_k, \Delta W_j))], \quad (7)$$

$$ContentScore_k = \frac{(1 + \beta_{fusion}^2) S_{consist,k} S_{contrib,k}}{\beta_{fusion}^2 S_{consist,k} + S_{contrib,k} + \varepsilon}. \quad (8)$$

With $\beta_{fusion} > 1$, the harmonic fusion gives greater emphasis to the clean direction while still requiring peer plausibility. Neither collusion nor deviation caused by Non-IID data can dominate the score alone.

4.2. Dual-Stream State Evolution

A single reputation value cannot distinguish low utility caused by data skew from high risk caused by an attack. Trust Flow keeps these effects in two streams: historical utility and short-term risk. Let $q_k^{(t)} = \text{clip}(ContentScore_k^{(t)}, 0, 1)$. The historical stream is an explicitly decayed Beta model:

$$\begin{aligned} a_k^{(t)} &= \beta_h a_k^{(t-1)} + q_k^{(t)}, \\ b_k^{(t)} &= \beta_h b_k^{(t-1)} + 1 - q_k^{(t)}, \\ HistPerf_k^{(t)} &= \frac{a_k^{(t)}}{a_k^{(t)} + b_k^{(t)} + \varepsilon}. \end{aligned} \quad (9)$$

For a fresh client, $a_k^{(0)} = b_k^{(0)} = 1$, so $HistPerf_k^{(0)} = 0.5$ rather than a presumed perfect reputation. The decay factor $\beta_h \in (0, 1)$ is fixed before test rounds. In the first round, a validly admitted client stays in NORMAL and receives only a provisional privilege based on current content and risk. That evidence is stored for the next state update; it cannot alone trigger an irreversible blacklist. Immediate control is therefore available through provisional weighting, while permanent decisions wait for cross-round evidence.

The history channel changes slowly: weak utility lowers privilege without immediately removing the client. The risk channel is faster, with $RiskEMA_k^{(t)} = \beta_r RiskEMA_k^{(t-1)} + (1 - \beta_r) Risk_k^{inst}$, where $Risk_k^{inst}$ is the maximum current probe. A strong warning cannot be averaged away by benign history.

Let $u_k^{(t)} = 1 - HistPerf_k^{(t)}$ and $r_k^{(t)} = RiskEMA_k^{(t)}$. The thresholds $(\tau_h, \tau_s, \tau_q, \tau_b) = (0.26, 0.64, 0.80, 0.90)$ govern utility deficit, soft risk, quarantine, and blacklisting. A NORMAL client moves to SUSPECT after two

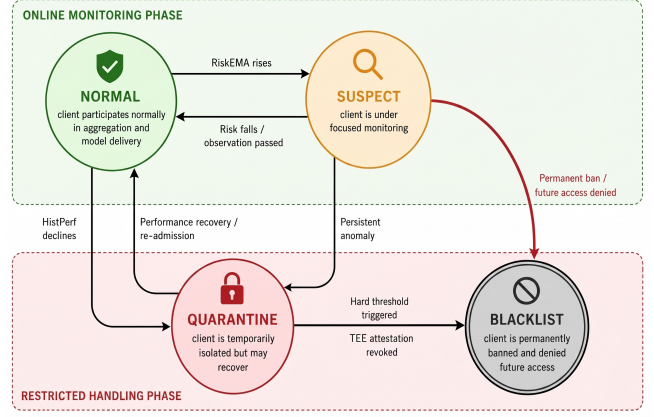


Figure 4. Client state-transition automaton driven by historical utility and short-term risk.

consecutive rounds with $u_k^{(t)} > \tau_h$ or $r_k^{(t)} \geq \tau_s$. From SUSPECT, three such rounds or one current probe with $Risk_k^{inst} \geq \tau_q$ cause QUARANTINE. A QUARANTINE client enters BLACKLIST after four consecutive rounds with $r_k^{(t)} \geq \tau_b$. SUSPECT and QUARANTINE recovery requires two consecutive rounds satisfying $u_k^{(t)} \leq \tau_h$ and $r_k^{(t)} < \tau_s$; BLACKLIST remains irreversible within an experiment. The relevant counters reset when their condition fails, and the state update finishes before layer aggregation. First-round control and permanent admission decisions consequently have different semantics: the former limits the current contribution, while the latter requires persistence.

The two streams meet only when computing aggregation privilege:

$$\begin{aligned} RawScore_k^{(t)} &= (TrustScore_k^{(t)})^\alpha \\ &\times (ContentScore_k^{(t)})^\beta (HistPerf_k^{(t)})^\gamma \\ &\times (1 - RiskEMA_k^{(t)})^p. \end{aligned} \quad (10)$$

The state transition is evaluated after the current probes and before aggregation. Current risk can therefore veto a sudden harmful burst even when historical utility is high, while a benign low-similarity client remains recoverable through the slow stream.

4.3. Risk-Gated Layered Aggregation

Backdoor effects often concentrate in deeper layers, while shallow feature layers may still contain useful heterogeneous information. Trust Flow assigns each layer l a sensitivity score. With L total layers, the three normalized components are

$$\begin{aligned} S_{priv}^{(l)} &= \exp(-\tau_p l / L), \quad S_{util}^{(l)} = \frac{\|g_{root}^{(l)}\|_2}{\max_m \|g_{root}^{(m)}\|_2 + \varepsilon}, \\ S_{sec}^{(l)} &= \frac{1}{2} \left(1 - \frac{1}{|\mathcal{T}|} \sum_{k \in \mathcal{T}} \cos(\Delta W_k^{(l)}, g_{root}^{(l)}) \right). \end{aligned} \quad (11)$$

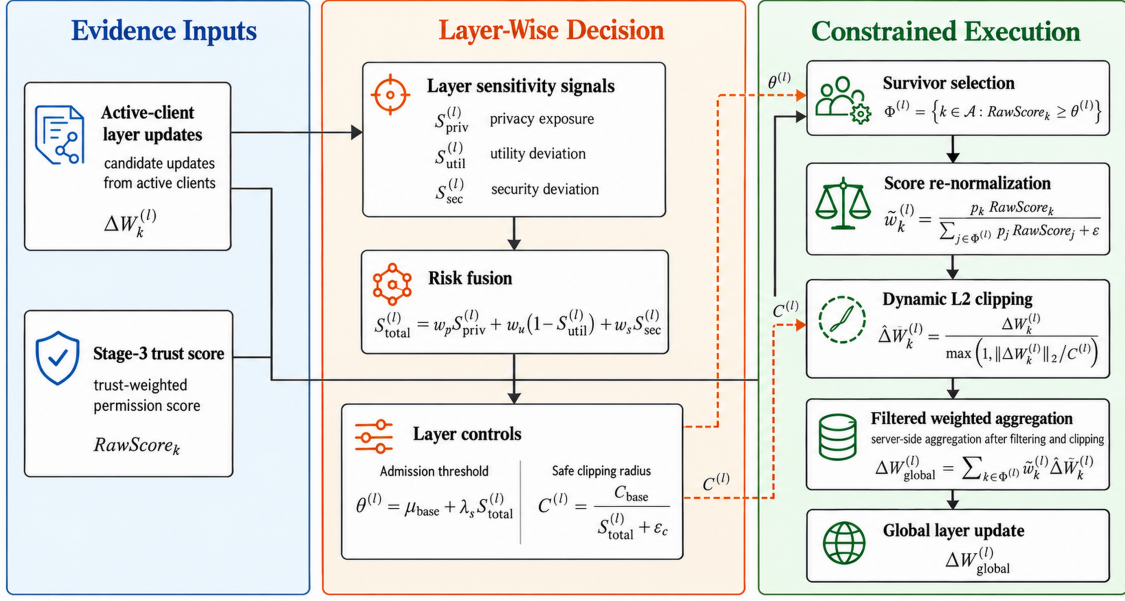


Figure 5. Risk-gated layer-wise clipping and differentiated aggregation.

Here $\mathcal{T} = \{k \in \mathcal{A} : M_{\text{attest},k} = 1, \text{RiskEMA}_k^{(t-1)} < \tau_s\}$ is the trusted reference set. If it is empty, the admitted set $\mathcal{A}$ is used only to normalize the layer score, and the absence of a trusted peer reference is recorded. Thus $S_{\text{priv}}^{(l)}, 1 - S_{\text{util}}^{(l)}, S_{\text{sec}}^{(l)} \in [0, 1]$. The implementation sets $S_{\text{total}}^{(l)} = w_p S_{\text{priv}}^{(l)} + w_u (1 - S_{\text{util}}^{(l)}) + w_s S_{\text{sec}}^{(l)}$, with $w_p, w_u, w_s = (0.3, 0.3, 0.4)$ and $w_p + w_u + w_s = 1$, so $S_{\text{total}}^{(l)} \in [0, 1]$. The threshold and clipping radius are

$$\theta^{(l)} = \mu_{\text{base}} + \lambda_s S_{\text{total}}^{(l)}, \quad C^{(l)} = \frac{C_{\text{base}}}{S_{\text{total}}^{(l)} + \varepsilon_c}. \quad (12)$$

A layer showing stronger suspicious drift therefore has a higher entry threshold and a smaller clipping radius. Layers that remain close to the reference retain more capacity for benign heterogeneity.

The settings are $\tau_p = 3.0$ and $\varepsilon = \varepsilon_c = 10^{-8}$. C_{base} is the median trusted-client update norm before layer gating. The sensitivity terms remain dimensionless, and the clipping scale follows the update magnitude of each round.

The surviving clients are $\Phi^{(l)} = \{k \in \mathcal{A} \mid \text{RawScore}_k^{(t)} \geq \theta^{(l)}\}$. Their weights are re-normalized:

$$\begin{aligned} \tilde{w}_k^{(l)} &= \frac{p_k \text{RawScore}_k^{(t)}}{\sum_{j \in \Phi^{(l)}} p_j \text{RawScore}_j^{(t)} + \varepsilon}, \\ \widehat{\Delta W}_k^{(l)} &= \frac{\Delta W_k^{(l)}}{\max(1, \|\Delta W_k^{(l)}\|_2 / C^{(l)})}. \end{aligned} \quad (13)$$

The final layer update is $\Delta W_{\text{global}}^{(l)} = \sum_{k \in \Phi^{(l)}} \tilde{w}_k^{(l)} \widehat{\Delta W}_k^{(l)}$. Re-normalization keeps the effective learning rate from shrinking merely because risky clients were removed.

If $\Phi^{(l)} = \emptyset$ because a sensitive layer rejects every candidate, the server keeps $k^* = \arg \max_{k \in \mathcal{A}} \text{RawScore}_k^{(t)}$ for that layer, gives it unit normalized weight, and still applies $C^{(l)}$. The layer update remains defined, while the threshold for other clients is unchanged.

Layer-wise control preserves useful shallow representations and applies stronger protection where suspicious drift is concentrated. A semantic backdoor that mainly alters deep features can be clipped or rejected there without imposing the same limit on benign shallow features. Once unsafe mass is removed, data-size-weighted surviving scores are projected back to a valid aggregation simplex, so filtering does not introduce an implicit learning-rate decay.

4.4. Risk Probe Computation

The short-term risk stream uses several intentionally redundant probes. For update k in round t , the normalized channels are

$$r_{\text{grad},k}^{(t)} = \text{clip}\left(\frac{1 - \cos(\Delta W_k, g_{\text{root}})}{2}, 0, 1\right), \quad (14)$$

$$r_{\text{amp},k}^{(t)} = \text{clip}\left(\frac{\|\Delta W_k\|_2}{\text{median}_j \|\Delta W_j\|_2 + \varepsilon} - 1, 0, 1\right), \quad (15)$$

$$r_{\text{clean},k}^{(t)} = \text{clip}\left(\frac{\mathcal{L}_{\text{probe}}(W + \Delta W_k) - \mathcal{L}_{\text{probe}}(W)}{\delta_{\text{clean}} + \varepsilon}, 0, 1\right), \quad (16)$$

$$r_{\text{trigger},k}^{(t)} = \text{clip}\left(\frac{\text{KL}(\mathcal{H}(A_{\text{base}}) \parallel \mathcal{H}(A_k))}{\delta_{\text{trigger}} + \varepsilon}, 0, 1\right), \quad (17)$$

$$\bar{c}_k^{(t)} = \frac{\sum_{j \neq k} \text{TrustScore}_j c_{kj}}{\sum_{j \neq k} \text{TrustScore}_j + \varepsilon},$$

$$r_{peer,k}^{(t)} = \text{clip}(1 - \bar{c}_k^{(t)}, 0, 1), \quad (18)$$

Here $c_{kj} = \cos(\Delta W_k, \Delta W_j)$. For the trigger probe, A_{base} is the activation histogram of the current global model on the server-held proxy set, and A_k is the histogram after candidate update k is applied. The histograms use 32 bins from the last convolutional block and the pre-classifier representation. In each round, δ_{clean} and $\delta_{trigger}$ equal the median plus three median absolute deviations of the corresponding values over $\mathcal{T}$, so calibration is independent of the candidate under test. Define $Risk_k^{inst} = \max\{1 - M_{attest,k}, r_{grad,k}, r_{amp,k}, r_{clean,k}, r_{trigger,k}, r_{peer,k}\}$.

The maximum keeps complementary warnings visible. Gradient and amplitude probes expose parameter attacks; clean-loss and activation probes address stealthier backdoors. Peer disagreement is supporting evidence because a benign long-tail update may differ from the majority. The execution order is reference construction, current-update audit, state update, and layer aggregation.

4.5. Assurance Composition and Reference Role

The four evidence sources have distinct responsibilities. Attestation and privacy-aware verification establish execution provenance and bounded statistics for admission. Cloud auditing examines the submitted update against reference or peer evidence. The temporal streams then decide whether the observation is transient or persistent. A valid measured process can therefore enter semantic auditing, while one low-similarity observation does not permanently remove a benign long-tail client.

The clean proxy acts as a reference initializer and calibration source; it does not replace the trust flow. It supplies a stable direction and a common probe interface when a small server-held set exists. Without that set, the peer-reference branch follows the same content and temporal sequence. In both cases, the final decision uses update evidence, state transitions, and layer-wise aggregation rather than treating one reference observation as a maliciousness label.

4.6. From Separate States to Participation Decisions

The two state variables answer different questions. $HistPerf$ records accumulated contribution, while $RiskEMA$ records recent risk. Weak historical utility can lower contribution and trigger reversible review, but the transition rule requires persistent high risk for irreversible blacklisting. Strong historical utility likewise does not cancel current risk. The policy thus separates useful-but-risky updates from low-utility-but-low-risk updates before assigning layer privileges.

For fixed admission factor, content score, and historical utility, write $B_k =$

$(TrustScore_k)^\alpha (ContentScore_k)^\beta (HistPerf_k)^\gamma$. With $p > 0$ and $0 \leq r < 1$, the score satisfies

$$\frac{\partial[B_k(1-r)^p]}{\partial r} = -pB_k(1-r)^{p-1} \leq 0, \quad (19)$$

$$r = RiskEMA_k^{(t)}.$$

At $r = 1$, the score is zero. This statement concerns the unnormalized score with the other factors fixed; layer selection, fallback, and normalization determine the update that is eventually applied. SUSPECT and QUARANTINE clients recover only under the stated consecutive-round utility and risk conditions. Low similarity alone is therefore not interpreted as maliciousness.

The mechanism-level evidence is separated across the ablations. Table 3 compares single-stream variants, HistPerf-only, and the complete dual stream on false bans and detection. Figure 7 isolates probe contributions and re-normalization. The first comparison tests the temporal policy; the second tests the evidence supplied to it and the optimization behavior after filtering.

5. Experiments

5.1. Setup

Experiments use Flower 1.5.0 and PyTorch on CIFAR-10 with a CIFAR-10-adapted ResNet-18. The first convolution uses a 3×3 kernel with stride one, the initial max-pooling layer is removed, and the classifier has ten outputs. Training is evaluated on paired host configurations, with the client execution path deployed in an Intel TDX confidential virtual machine. NVIDIA A10 GPUs execute model training. The TDX guest protects attestation, trusted monitoring, report generation, and admission-state processing. The TDX procedure obtains a TDREPORT, converts it into a remotely verifiable Quote, checks nonce freshness and measured-code identity, and binds the round identifier and update digest before admission. The testbed uses anonymized paired host configurations and five NVIDIA A10 GPUs. The software stack is Ubuntu 20.04.6 LTS and CUDA 12.8. The 50,000 training images are partitioned into a shared 5,000-image client pool, replicated as a common base subset for all clients, and 45,000 exclusive images. Clients 0–9 use an IID split, clients 10–14 use Dirichlet $\alpha = 1.0$, and clients 15–19 use Dirichlet $\alpha = 0.1$. Separately, the server holds a balanced 500-image proxy set, with 50 images per class, from the held-out CIFAR-10 reference partition. It is disjoint from client training and final evaluation. The fixed proxy partition and initialization are shared across all methods. Reference-dependent methods use it for their defined guiding operation and probes, while methods without a reference channel retain their standard aggregation rule. All methods use the same optimizer, communication budget, client partition, and initial model. Five independent seeds are run on each hardware configuration, and means and standard deviations are computed over the resulting ten platform-seed runs.

The formal 30% attack setting uses six malicious clients. Client 0 performs untargeted label flipping with $\gamma = 0.5$. Client 1 injects a fixed 3×3 lower-right trigger into 20% of local samples and targets class 0. Client 2 uses label-preserving clean-label perturbations on 20% of local samples with the same target. Client 3 applies a fixed semantic attribute bias to 20% of local samples, with the biased subset assigned to class 0. Client 4 applies untargeted sign flipping, and Client 5 applies untargeted gradient scaling by a factor of 100. Clients 1–3 are evaluated as targeted attacks, whereas Clients 0, 4, and 5 are evaluated through clean loss, convergence, and detection state. Attack behaviors are active in every local training round; no attack is introduced only after observing the global trajectory. Five Sybil nodes without valid certificates and three free-rider nodes with forged low workload are used only to validate Phase-One gating. These eight nodes are intercepted before statistical aggregation, so later metrics are computed over the 20 admitted clients. Malicious updates that pass hardware attestation represent an attacker operating inside an admissible training process; post-attestation host tampering is excluded by the stated threat model. Each run uses 30 rounds and three local epochs. Aggregate values are averaged over the pooled platform-seed runs; state trajectories use one fixed seed per platform.

Clean accuracy and targeted ASR are reported as mean and standard deviation over the ten platform-seed runs. For each targeted behavior $a \in \{\text{trigger, clean, semantic}\}$, let $\mathcal{B}_a$ be its trigger- or attribute-bearing test subset and y_a^* its declared target. Define $ASR_a = |\{x \in \mathcal{B}_a : \arg \max f(x) = y_a^*\}|/|\mathcal{B}_a|$ and report the macro-average $ASR_{target} = \frac{1}{3} \sum_a ASR_a$. The three attack-specific quantities remain separate in the convergence traces; the headline table uses the macro-average so that no targeted behavior receives extra weight. Untargeted label/sign/scaling behavior is assessed through clean accuracy, loss, and state-transition time rather than this numerator. Permanent-ban FPR counts only benign clients that are irreversibly blacklisted; temporary SUSPECT and QUARANTINE states are recoverable controls. Stateless baselines have no comparable permanent-ban state, so their FPR is reported as N/A. All methods share the same client partition, initial model, optimizer, communication budget, attack schedule, reference availability, and Non-IID split; each applies its own defined decision rule to the shared evidence.

5.2. Overall Defense Performance

Figure 6 collects convergence, the three attack-specific ASR traces, trust-state trajectories, and feature separation. In the stated 30% mixed-attack protocol, all six attack behaviors run concurrently in every local round; the experiment is therefore a simultaneous-attack test. Trust Flow reaches 92.31% clean accuracy and 10.21% macro-averaged targeted ASR, close to the 10% random-target rate for ten CIFAR-10 classes. The state traces show the division of labor among the assurance layers. Execution-valid clients remain eligible for semantic auditing, current probes expose harmful update

TABLE 1. EXPERIMENTAL CONFIGURATION AND MAIN HYPERPARAMETERS.

Category	Setting	Value
FL task	Dataset/model/admitted clients	CIFAR-10/CIFAR-adapted ResNet-18/ $K = 20$
Optimization	Local epochs, lr, momentum, rounds	3, 0.001, 0.9, 30
Data heterogeneity	Shared client pool; server proxy; Group A/B/C	5000; 500; $\alpha = 1.0, 0.1$
Trust streams	$\alpha_1, \alpha_2, \beta_{fusion}; \beta_h, \beta_r$	0.6, 0.4, 2.0; 0.9, 0.85
Admission filter	$w_a, \lambda, Q;$ $T_0, P_0, RiskEMA_0$	0.7, 1.0, 0.01; 1, 0.25, 0
Layer gating	$\alpha, \beta, \gamma, p; \mu_{base}, \lambda_s$	3.0, 1.0, 0.5, 1.0; 0.4, 1.2
Sensitivity	$\tau_p; w_p, w_u, w_s$	3.0; 0.3, 0.3, 0.4
Numerical stability	$\varepsilon, \varepsilon_c; C_{base}$	$10^{-8}, 10^{-8}$; median
State machine	$\tau_h, \tau_s, \tau_q, \tau_b;$ persistence	0.26, 0.64, 0.80, 0.90; 2, 3, 4 rounds

behavior, and the temporal streams determine whether an observation is transient or persistent. A legitimate long-tail client may receive a provisional restriction while retaining its history, whereas persistent risk drives the specified state transitions. These measurements describe the stated protocol and the evidence interfaces shared by the compared methods.

The trajectories retain elevated risk for label, trigger, semantic, sign, and scaling attacks. A benign long-tail client can recover when its risk does not persist. The t-SNE panel visualizes the resulting trust-state features; quantitative claims come from accuracy, ASR, and state-transition metrics.

In the matched classical comparison, Krum reaches 64.20% accuracy, Trimmed Mean 71.35%, and FLTrust 81.25%, while Trust Flow reaches 92.31%. These values do not establish a universal ranking. DeepSight, CrowdGuard, RoseAgg, FLPurifier, ShieldFL, and RaSA require attack-specific or privacy-preserving interfaces that are not reproduced here, so their absence is a comparison limitation. The 50% result remains an out-of-assumption stress test rather than evidence of a theoretical Byzantine boundary.

5.3. Ablation, Sensitivity, and Overhead

Table 3 makes the single-stream FPR/TPR tradeoff visible and shows the low recall of HistPerf-only. The reported TrustReport size is about 15 KB, and server aggregation plus audit increases from 2.10 s to 4.85 s. Optional cryptographic secure aggregation remains outside the stated system boundary.

The server timing covers aggregation and content audit. The measured auxiliary security-path overhead is approximately 11 ms per admitted-client round. Device-side TEE Quote generation, signing, and lightweight monitoring are measured separately at approximately 12–15 ms per admitted-client round. These values describe their respective measurement scopes rather than one end-to-end deployment

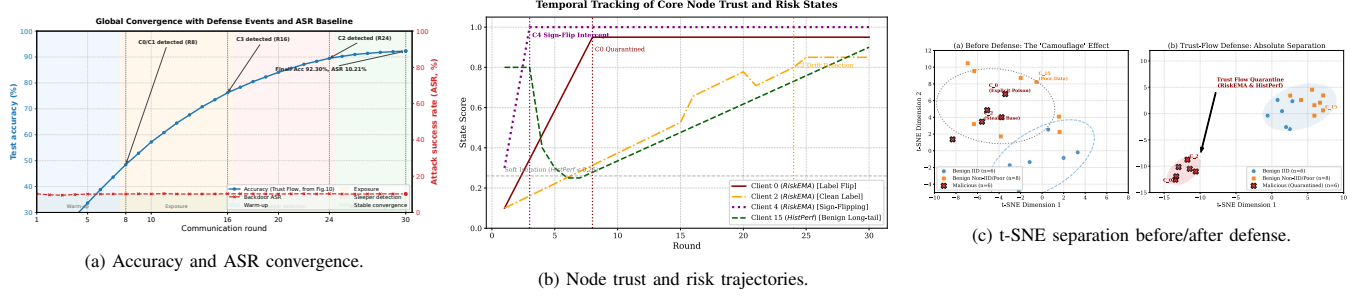


Figure 6. Main defense behavior under mixed attacks.

TABLE 2. PERFORMANCE COMPARISON UNDER 30% MIXED ATTACKS AND BOUNDARY STRESS TESTING.

Method/Setting	Core mechanism	Acc.	Targeted ASR	Perm. FPR [†]
FedAvg [2]	no defense	86.15 $\pm$ 1.20	92.34 $\pm$ 5.10	N/A
Krum [3]	Euclidean selection	64.20 $\pm$ 2.80	12.15 $\pm$ 0.90	N/A
Trimmed Mean [4]	coordinate trimming	71.35 $\pm$ 2.40	38.60 $\pm$ 4.20	N/A
FLTrust [5]	clean anchor weighting	81.25 $\pm$ 1.50	15.42 $\pm$ 1.10	N/A
FLAME [9]	clustering, clipping, noise	73.18 $\pm$ 1.80	19.00 $\pm$ 0.60	N/A
FLDetector [7]	update consistency	65.89 $\pm$ 2.30	23.00 $\pm$ 1.20	N/A
FreqFed [22]	frequency analysis	79.37 $\pm$ 0.90	14.00 $\pm$ 1.10	N/A
Trust Flow	dual stream + layered gating	92.31 $\pm$ 0.09	10.21 $\pm$ 0.05	0.00
Boundary stress: 50% malicious	10/20 malicious	91.81 $\pm$ 0.10	10.46 $\pm$ 0.06	0.00

[†] Permanent-ban FPR is defined only for Trust Flow and its stateful ablations. Krum, Trimmed Mean, and FLTrust are stateless in this comparison and are therefore marked N/A. Temporary SUSPECT/QUARANTINE states are recoverable and reported separately through trajectories.

TABLE 3. ABLATION AND OVERHEAD SUMMARY.

Study	Variant / architecture	FPR or overhead	TPR / time
Reputation	Single-stream EMA, aggressive	18.25% FPR	95.12% TPR
Reputation	Single-stream EMA, conservative	2.10% FPR	48.33% TPR
Reputation	HistPerf-only	0.15% FPR	21.05% TPR
Reputation	Dual-stream Trust Flow	0.00% FPR	100.00% TPR
Overhead	Standard FedAvg	0 KB extra	2.10 s aggregation
Overhead	Trust Flow	~15 KB TrustReport	4.85 s incl. audit

latency. CPU-side TDX isolation does not by itself establish protection of GPU execution or GPU memory.

Figures 7 and 8 show that treating SUSPECT as recoverable review yields 88.33% per-round malicious coverage and a 14.79% benign review rate, with no permanent false ban in the tested runs. Activation and clean-loss probes reduce the reported clean-label ASR below 9.2%; re-normalization avoids an implicit learning-rate decay. At $\alpha = 0.1$, Trust Flow remains near 92% accuracy across the tested Intel TDX configurations. These ablation values are descriptive measurements under the same protocol and do not claim statistical significance beyond the repeated-run summary.

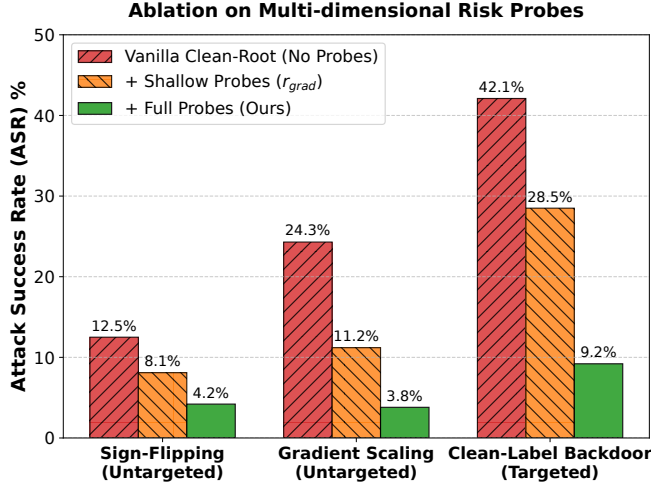
The Dirichlet sweep in Fig. 8 changes only the con-

centration parameter, while label space, model, optimizer, and attack protocol stay fixed. As α decreases, Krum and FLTrust increasingly reject legitimate drift. Trust Flow is less sensitive in the reported sweep because content is combined with history and risk. The default $\beta_{fusion} = 2.0$ gives the best observed accuracy–ASR balance. The reported 2.10 s to 4.85 s runtime covers server-side aggregation and auditing, not end-to-end TDX deployment latency.

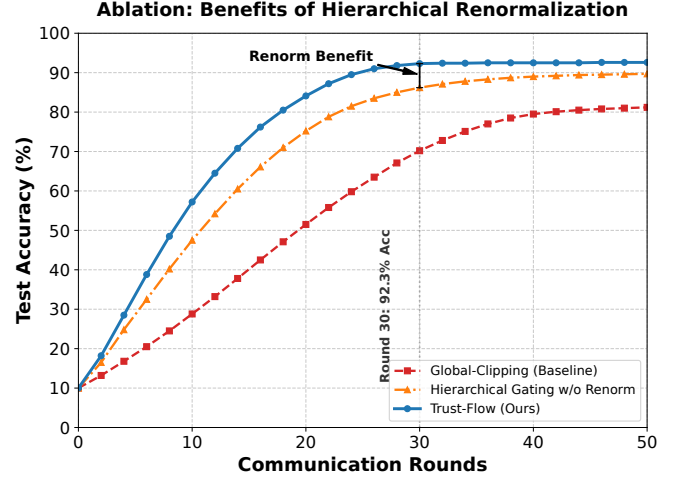
6. Discussion and Conclusion

Trust Flow places execution admission, privacy-aware statistical checks, content auditing, temporal reputation, and layer-wise aggregation in one trust-state pipeline. Its system-level contribution lies in the ordering of these evidence streams. Execution evidence controls admission; privacy-aware checks expose bounded data evidence; current probes control immediate risk; historical utility controls recoverable retention; and layer scores determine where an update can contribute. On the Intel TDX-backed execution path and paired host configuration, this separation keeps permanent-ban FPR near zero while retaining high recall against the tested mixed attacks. Attestation establishes execution eligibility, whereas cloud-side content and temporal evidence determine update safety.

The evaluated profile is server-assisted vision FL with a small trusted reference partition and an attestation-capable client path. The peer-reference branch follows the same



(a) Multi-dimensional risk-probe ablation.



(b) Weight re-normalization effect.

Figure 7. Component-level ablation of risk probes and re-normalization.

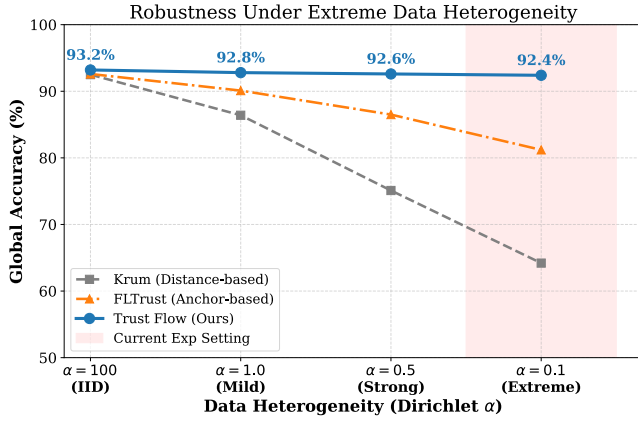


Figure 8. Robustness stress test under varying data heterogeneity.

decision sequence when no server proxy is available, while the measurements here use the common reference-initialized path. Microarchitectural side channels and confidential-VM resource contention remain outside the threat model. The probes are tailored to vision classification; report size and server audit overhead are kept separate from end-to-end latency. Cryptographic secure aggregation is outside the main boundary because the server must inspect individual evidence. This profile states where the trust-flow contract applies and provides a basis for extending it to other data modalities and deployment conditions.

References

- [1] P. Kairouz, H. B. McMahan, B. Avent *et al.*, “Advances and open problems in federated learning,” *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [2] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agueria y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, vol. 54, 2017, pp. 1273–1282. [Online]. Available: <https://proceedings.mlr.press/v54/mcmahan17a.html>
- [3] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, “Machine learning with adversaries: Byzantine tolerant gradient descent,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: <https://papers.nips.cc/paper/6617-machine-learning-with-adversaries-byzantine-tolerant-gradient-descent>
- [4] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, “Byzantine-robust distributed learning: Towards optimal statistical rates,” in *Proceedings of the 35th International Conference on Machine Learning*, vol. 80, 2018, pp. 5650–5659. [Online]. Available: <https://proceedings.mlr.press/v80/yin18a.html>
- [5] X. Cao, M. Fang, J. Liu, and N. Z. Gong, “FLTrust: Byzantine-robust federated learning via trust bootstrapping,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2021.
- [6] C. Fung, C. J. M. Yoon, and I. Beschastnikh, “The limitations of federated learning in Sybil settings,” in *Proceedings of the 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2020, pp. 301–316. [Online]. Available: <https://www.usenix.org/conference/raid2020/presentation/fung>
- [7] Z. Zhang, X. Cao, J. Jia, and N. Z. Gong, “FLDetector: Defending federated learning against model poisoning attacks via detecting malicious clients,” in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022, pp. 2545–2555.
- [8] P. Rieger, T. D. Nguyen, M. Miettinen, and A. R. Sadeghi, “Deep-Sight: Mitigating backdoor attacks in federated learning through deep model inspection,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2022.
- [9] T. D. Nguyen, P. Rieger, H. Chen, H. Yalame, H. Möllering, H. Fereidooni, S. Marchal, M. Miettinen, A. Mirhoseini, S. Zeitouni, F. Koushanfar, A.-R. Sadeghi, and T. Schneider, “FLAME: Taming backdoors in federated learning,” in *Proceedings of the 31st USENIX Security Symposium*, 2022, pp. 1415–1432. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/nguyen>
- [10] P. Rieger, T. Krauß, M. Miettinen, A. Dmitrienko, and A. R. Sadeghi, “CrowdGuard: Federated backdoor detection in federated learning,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2024.

- [11] H. Yang, W. Xi, Y. Shen, C. Wu, and J. Zhao, "RoseAgg: Robust defense against targeted collusion attacks in federated learning," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 2951–2966, 2024.
- [12] Y. Chen, F. Luo, T. Li, T. Xiang, Z. Liu, and J. Li, "A training-integrity privacy-preserving federated learning scheme with trusted execution environment," *Information Sciences*, vol. 522, pp. 69–79, 2020.
- [13] T. Xu, K. Zhu, A. Andrzejak, and L. Zhang, "Distributed learning in trusted execution environment: A case study of federated learning in SGX," in *Proceedings of the 7th IEEE International Conference on Network Intelligence and Digital Content (IC-NIDC)*, 2021, pp. 450–454.
- [14] S. Queyrut, V. Schiavoni, and P. Felber, "Mitigating adversarial attacks in federated learning with trusted execution environments," in *Proceedings of the IEEE 43rd International Conference on Distributed Computing Systems (ICDCS)*, 2023, pp. 626–637.
- [15] Z. Lu, L. Wang, Z. Zhang, M. Huang, J. Wang, and M. Li, "TMT-FL: Enabling trustworthy model training of federated learning with malicious participants," *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 3, pp. 2723–2740, 2025.
- [16] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How To Backdoor Federated Learning," in *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, vol. 108, 2020, pp. 2938–2948. [Online]. Available: <https://proceedings.mlr.press/v108/bagdasaryan20a.html>
- [17] B. Wang, Y. Yao, S. Shan, H. Li, B. Viswanath, H. Zheng, and B. Y. Zhao, "Neural Cleanse: Identifying and mitigating backdoor attacks in neural networks," in *Proceedings of the IEEE Symposium on Security and Privacy (SP)*, 2019, pp. 707–723.
- [18] J. Zhang, C. Zhu, X. Sun, C. Ge, B. Chen, W. Susilo, and S. Yu, "FLPurifier: Backdoor defense in federated learning via decoupled contrastive training," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 4752–4766, 2024.
- [19] Z. Ma, J. Ma, Y. Miao, Y. Li, and R. H. Deng, "ShieldFL: Mitigating model poisoning attacks in privacy-preserving federated learning," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 1639–1654, 2022.
- [20] L. Wang, M. Huang, Z. Zhang, M. Li, J. Wang, and K. Gai, "RaSA: Robust and adaptive secure aggregation for edge-assisted hierarchical federated learning," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 4280–4295, 2025.
- [21] X. Zhang, Z. Chen, G. Chen, X. Feng, Q. Shen, and Z. Wu, "RPPFL: Robust and privacy-preserving federated learning via trusted execution environments," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025, pp. 1–5.
- [22] H. Fereidooni, A. Pegoraro, P. Rieger, A. Dmitrienko, and A.-R. Sadeghi, "FreqFed: A frequency analysis-based approach for mitigating poisoning attacks in federated learning," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2024. [Online]. Available: <https://doi.org/10.14722/ndss.2024.23620>